EMBEDDED SYSTEMS • MINI PROJECT DOCUMENTATION

The Digital Clock

Built Around the Bare ATmega328P-PU Microcontroller

From Breadboard Prototype to a 3D-Printed, Snap-Fit Enclosure – a Complete Idea-to-Product Embedded Engineering Workflow

Microcontroller:	ATmega328P-PU (standalone, no Arduino board)
Display:	4-Digit Multiplexed Seven-Segment (HH:MM / MM:SS)
Programming Tool:	Arduino Uno used as an AVR ISP Programmer
Enclosure:	Fusion 360 CAD, Snap-Fit, FDM 3D Printed

Project Documentation Report

Department of Electronics & Communication Engineering

June 19, 2026

Abstract

This report documents the complete engineering workflow behind a **standalone digital clock** built around the bare **ATmega328P-PU** microcontroller – the same chip that powers the Arduino Uno, used here without the Arduino board itself. The project follows a deliberate, industry-style path: requirement-driven **component selection**, supporting **engineering calculations** (crystal load capacitance, segment current-limiting resistors, decoupling and reset networks), a **breadboard prototype on an Arduino Uno** to validate display multiplexing and button logic, migration to the **bare ATmega328P-PU chip**, **bootloader burning** using a second Arduino Uno configured as an AVR ISP programmer, **firmware upload**, full-circuit **breadboard re-validation**, permanent **soldering**, and finally a custom **snap-fit enclosure** modelled in Autodesk Fusion 360 and 3D printed after slicing in Creality's slicer software. Every design decision – from the 22 pF crystal capacitors to the 330 Ω segment resistors – is backed by a calculation, making this document a complete reference for anyone wishing to reproduce, audit, or extend the build.

Keywords: ATmega328P-PU · AVR ISP · 7-Segment Multiplexing · Crystal Oscillator · Bootloader Burning · Fusion 360 · Snap-Fit Enclosure · FDM 3D Printing

Contents

1 Introduction

1.1 Motivation

Most beginner digital-clock projects stop at "upload code to an Arduino board and watch the display blink." That is a fine starting point, but it stops short of the question every embedded engineer eventually has to answer: *what happens once you remove the development board?* An Arduino Uno is, underneath the USB connector, the voltage regulator, and the FTDI/CH340 chip, simply a breakout board for one microcontroller – the **ATmega328P**. This project was undertaken to deliberately strip away that convenience layer and rebuild the same functionality around the bare chip, going through every step a product team would: selecting parts for a reason, calculating support-component values instead of guessing them, validating logic on a breadboard, porting the firmware to the standalone microcontroller, burning a bootloader, soldering a permanent board, and finally packaging the result in a custom-designed, 3D-printed enclosure.

1.2 Project Objective

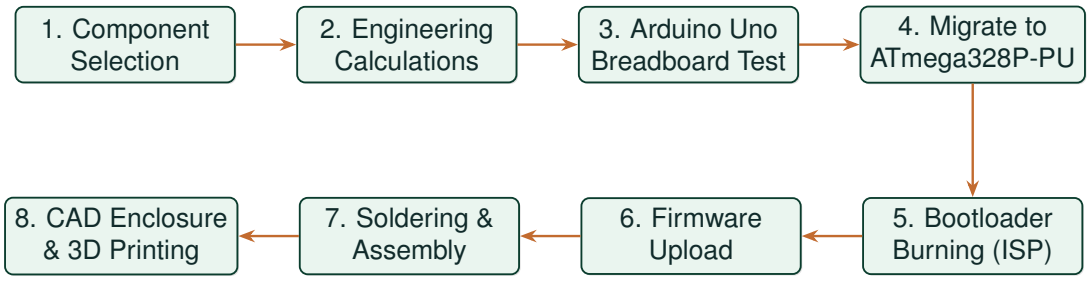
Objective

To design and implement a **standalone digital clock** using an **ATmega328P-PU** microcontroller (rather than a complete Arduino board), demonstrating the full embedded-product development pipeline – from idea to a finished, enclosed device – and to document every design choice with the calculation that justifies it.

The final device displays the time on four multiplexed seven-segment displays and accepts user input through three push buttons, with two operating modes (*Normal* and *Seconds*) described in Section 2.

1.3 Development Workflow at a Glance

The project was executed as eight sequential phases, each building on the validation done in the one before it. This mirrors how a real embedded product moves from concept to manufacture: nothing is soldered until it has been proven on a breadboard, and nothing is enclosed until the soldered board itself is verified.



Why this order matters

Logic is validated on the **Arduino Uno** first because its USB bootloader and on-board regulator make iteration fast – mistakes there cost a re-upload, not a re-solder. Only after the segment mapping, multiplexing, and button logic are confirmed correct does the design move to the bare chip, where mistakes are costlier to fix.

2 Final Feature Set

The firmware implements two display modes and a simple three-button control scheme, chosen to keep the user interface minimal while still allowing full time-setting capability.

2.1 Normal Mode — HH:MM

This is the default, always-returned-to display mode. The four digits show hours and minutes, separated by a colon built from two LEDs, e.g.:

14:35

2.2 Seconds Mode — MM:SS

A long press of the *Start/Mode* button switches the display to minutes and seconds:

35:27

After a short timeout, the firmware automatically reverts to Normal Mode, so the clock never gets "stuck" showing seconds.

2.3 User Controls

Three momentary push buttons, wired with internal pull-ups, provide the entire user interface:

Button	Label	Function
Button 1	HOUR	Increment the hour value
Button 2	MINUTE	Increment the minute value
Button 3	START/MODE	Start the clock from a set time / toggle Seconds Mode

Active-Low Button Logic

With the internal pull-up resistors enabled in firmware, each button pin idles **HIGH**. Pressing a button pulls the pin to **LOW** (ground). The firmware therefore treats a logic-0 read as "pressed" – no external pull-up/pull-down resistors are required on the board.

3 System Architecture

At the block level, the clock has exactly four functional groups: a clock-generation source, the microcontroller itself, the output display (digits and colon), and the input (buttons). Figure 1 shows how they connect.

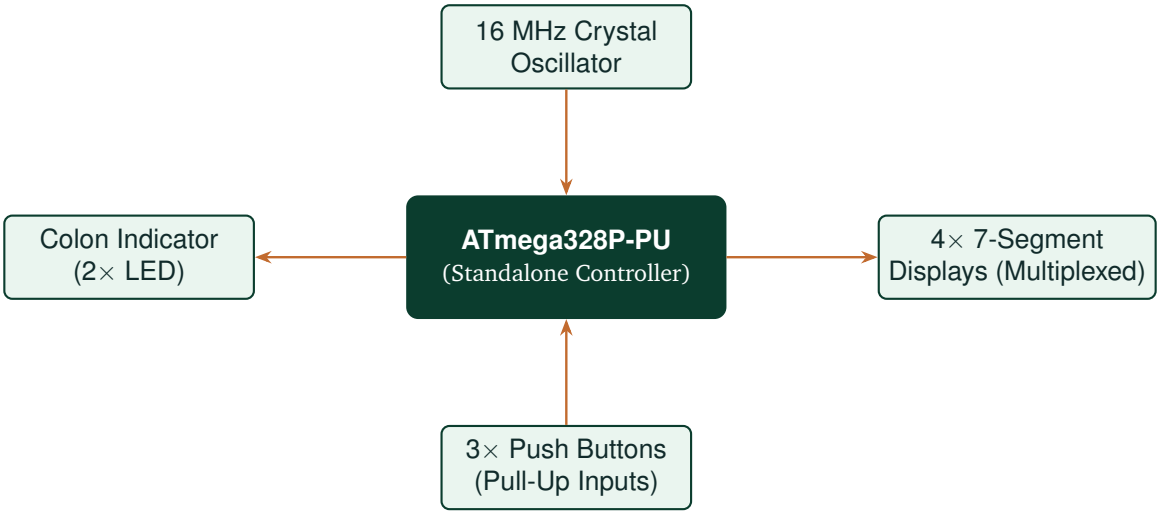


Figure 1: System block diagram: the ATmega328P-PU sits at the centre, clocked by a crystal, reading three buttons, and driving the multiplexed seven-segment display together with the colon LEDs.

Why no separate driver ICs?

With only 4 digits and 7 segments (11 signal lines) plus 2 colon LEDs and 3 buttons, the entire interface fits comfortably within the ATmega328P-PU's 23 usable GPIO pins (Section 7.2), so no external display-driver or multiplexer IC was needed – the microcontroller drives everything directly.

4 Component Selection & Engineering Rationale

Every part on the bill of materials was chosen to satisfy a specific requirement of the system – nothing was added "because it's commonly used." Table 1 walks through that reasoning component by component.

4.1 Selection Criteria

Requirement	Component Chosen	Engineering Rationale
Processing core	ATmega328P-PU (DIP-28)	Same silicon as the Arduino Uno, but ~80% cheaper alone; allows a compact, board-free final product.
System clock	16 MHz quartz crystal	Required by the Arduino core's timing libraries (`millis()`, `delay()`); the internal 8 MHz RC oscillator is both slower and far less accurate for long-term timekeeping.
Crystal loading	2× 22 pF ceramic capacitors	Brings the effective load capacitance to the crystal's specified range (Section 5.1).
Supply stabilisation	2× 100 nF ceramic capacitors	Suppresses switching noise on VCC/AVCC, preventing erratic resets.
Reset stability	10 kΩ pull-up resistor	Holds RESET firmly HIGH so it cannot float and re-trigger the chip.
Time display	4× common-anode 7-segment displays	Inexpensive, widely available, and well-suited to multiplexed driving from a handful of GPIO pins.
Segment current limiting	330 Ω resistors	Calculated (Section 5.4) to hold segment current near 10 mA for safe, bright operation.
HH:MM / MM:SS separator	2× LED + 330 Ω resistors	Cheaper and simpler than a fifth digit; clearly marks the colon position.
User input	3× tactile push buttons	Minimum number of controls needed for hour-set, minute-set, and mode/start; internal pull-ups remove the need for external resistors.
Firmware burning	A second Arduino Uno as ISP	Avoids purchasing a dedicated USBasp/AVRISP programmer – a Uno already on hand does the job.
Power input	5 V USB power module	Ubiquitous, safe low-voltage supply compatible with any USB charger or power bank.
Enclosure	Fusion 360-designed, FDM-printed, snap-fit two-part shell	Protects the soldered board and gives a finished look without screws or extra hardware.

Table 1: Component selection criteria and the reasoning behind each choice.

4.2 Bill of Materials

Component	Specification	Qty	Purpose
ATmega328P-PU	DIP-28, 16 MHz rated	1	Main controller
Crystal oscillator	16 MHz, HC-49 package	1	System clock source
Ceramic capacitor	22 pF	2	Crystal load capacitors
Ceramic capacitor	100 nF	2	Decoupling: VCC & AVCC
Resistor	10 kΩ	1	RESET pull-up
Resistor	330 Ω	9	7 segment lines (a–g) + 2 colon LEDs
7-Segment display	Common anode, single digit	4	Time display (multiplexed)
LED (colon indicator)	3 mm/5 mm, any colour	2	HH:MM / MM:SS separator
Tactile push button	6 mm through-hole	3	Hour / Minute / Start-Mode input
DIP-28 IC socket	0.3″ pitch (optional)	1	Allows chip removal without re-soldering
USB power module	5 V regulated output	1	Power supply
Perforated/dot PCB	Standard prototyping board	1	Final permanent assembly
Hook-up / jumper wire	22–24 AWG	As needed	Breadboard and solder-side wiring
PLA filament	1.75 mm, FDM printed	≈ 1 spool	Enclosure (top + bottom shell)

Table 2: Final bill of materials for the standalone clock.

■ Tools used but not part of the final product

A breadboard, a second **Arduino Uno** (acting purely as an AVR ISP programmer), a soldering iron with fine tip, a digital multimeter, and a 3D printer were all essential to building this project but do not appear in the BOM above because they are not consumed by, or shipped with, the final device.

5 Design Calculations

This section derives every passive-component value used in the standalone circuit. None of the values were copied from an unrelated reference design – each follows from a short calculation given below.

5.1 Clock Source: Crystal Load Capacitance

The ATmega328P-PU needs a stable timing reference for accurate second-counting. A 16 MHz quartz crystal is wired across the `XTAL1/XTAL2` pins, each leg also returned to ground through a capacitor (Figure 2).

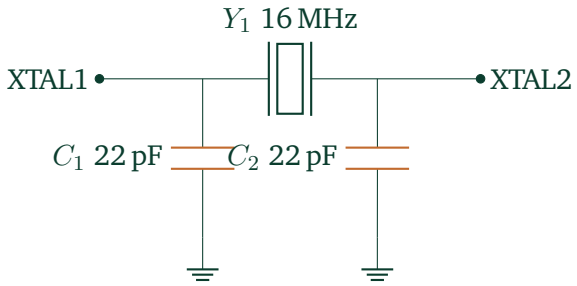


Figure 2: Crystal oscillator network: a 16 MHz crystal with two 22 pF load capacitors returned to ground.

Load Capacitance Calculation

The crystal's effective load capacitance is:

$$C_L = \frac{C_1 C_2}{C_1 + C_2} + C_{\text{stray}}$$

With $C_1 = C_2 = 22\,\text{pF}$:

$$\frac{C_1 C_2}{C_1 + C_2} = \frac{22 \times 22}{22 + 22} = 11\,\text{pF}$$

Adding a typical board/breadboard stray capacitance of $C_{\text{stray}} \approx 4$–$5\,\text{pF}$ gives:

$$C_L \approx 15\text{–}16\,\text{pF}$$

This sits inside the load-capacitance window most 16 MHz AVR-grade crystals are specified for, which is why **22 pF** is the standard, datasheet-recommended capacitor value for this oscillator circuit – and why it was chosen here rather than a smaller or larger value.

5.2 Decoupling Capacitor Network

Two 100 nF ceramic capacitors bypass the digital supply (`VCC`) and the analog supply (`AVCC`) directly to ground, placed as close to the chip's pins as practical (Figure 3).

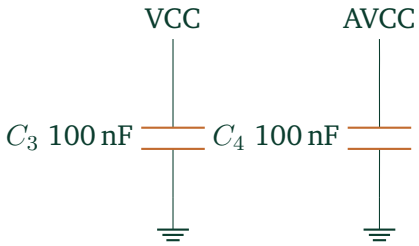


Figure 3: Decoupling network on VCC and AVCC.

Why this matters

Every time the multiplexer routine switches a digit or the firmware toggles a segment, the chip's internal switching draws a brief current spike. Without local decoupling, that spike shows up as a voltage dip on the supply rail – large enough, in some cases, to dip below the chip's minimum operating voltage and cause a spurious reset. The 100 nF capacitors act as tiny local reservoirs that supply this instantaneous current, keeping the rail flat.

5.3 Reset Pull-Up Network

The RESET pin is active-low: pulling it to ground resets the chip. Left unconnected, it is a high-impedance node that can be pulled low by noise alone. A 10 kΩ resistor to 5V (Figure 4) holds it firmly HIGH during normal operation.

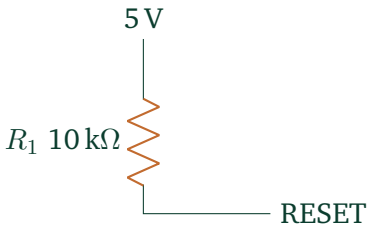


Figure 4: External RESET pull-up resistor.

5.4 Segment Current-Limiting Resistor

Each seven-segment LED needs a series resistor so the current through it stays in a safe, consistently bright range. Figure 5 shows the arrangement for one segment line; the same resistor value is reused for all seven segment lines (a–g) since they share identical electrical characteristics.

Figure 5: Current-limiting resistor in series with one display segment.

Ohm's Law Resistor Sizing

$$R = \frac{V_{\text{supply}} - V_f}{I_{\text{segment}}}$$

With a 5 V supply, a typical red-LED segment forward drop $V_f \approx 2\,\text{V}$, and a target segment current of $I = 10\,\text{mA}$:

$$R = \frac{5 - 2}{0.01} = 300\,\Omega$$

The closest standard (E12 series) resistor value *above* the calculated minimum is:

$$\boxed{R = 330\,\Omega}$$

Power Budget Check

With 7 segments lit at ∼10 mA each, a fully-lit digit (e.g. "8") momentarily draws up to $7 \times 10\,\text{mA} = 70\,\text{mA}$ through its active digit line. This is comfortably below the ATmega328P's absolute device-level current budget, but it is *above* the recommended continuous 20 mA-per-pin guideline for a single AVR I/O pin. Because the display is multiplexed – only one digit is ever fully lit at an instant, each for a few milliseconds – the *average* current per pin over a full refresh cycle is far lower than this peak. As good practice for any future revision, routing the digit-select (common) lines through a small NPN/PNP transistor or low-R_{DS} MOSFET per digit – rather than driving them straight from the microcontroller – keeps every AVR pin safely inside its rated current and is the standard approach in commercial multiplexed-display products.

6 Phase I — Breadboard Prototyping on the Arduino Uno

6.1 Why Prototype Before Going Standalone

Before touching a bare chip, the complete display and button logic was built and tested on a breadboard **using a full Arduino Uno**. The goal of this phase was purely to validate software and wiring logic while iteration was still cheap:

- Correct segment-to-digit mapping (does "2" actually look like a 2 on every digit?)
- Correct multiplexing refresh rate (no visible flicker, no ghosting between digits)
- Correct button debounce and pull-up logic
- Correct `millis()`-based timekeeping (no drift, no missed seconds)

Any mistake at this stage costs nothing more than a re-upload. The same mistake discovered after soldering the standalone chip costs a desoldering iron and a replacement part.

6.2 Arduino Uno Pin Allocation

Arduino Pins	Connected To	Role
D2 – D8	Segments a, b, c, d, e, f, g	Shared segment lines (all 4 digits)
D9 – D12	Digit 1, 2, 3, 4 common lines	Digit (multiplex) select
A0 – A2	Button 1, 2, 3	Hour / Minute / Start-Mode input

Table 3: Pin allocation used during the Arduino Uno breadboard prototype.

6.3 Display Multiplexing Principle

Driving four seven-segment digits independently would need $4 \times 7 = 28$ segment-control pins – far more than is practical. Instead, all four digits' corresponding segments are wired together onto just seven shared lines (a through g), and a separate "digit select" line determines *which one digit* is currently allowed to light up (Figure 6).

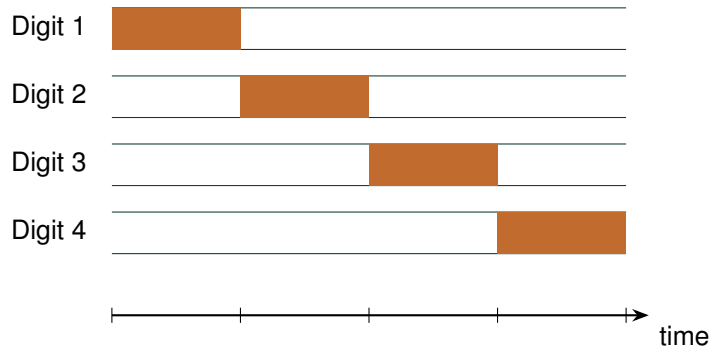


Figure 6: Multiplexing timeline: only one digit's common line is active at any instant, cycling rapidly through all four.

Persistence of Vision

The human eye cannot distinguish individual flashes faster than roughly 50–60 times per second; it perceives them as continuously lit. By switching through all four digits well above this rate, the firmware shows only one physical digit at a time, yet the viewer perceives a steady, complete `14:35` on the display.

6.4 Button Interfacing

The three push buttons reuse the same active-low, internal-pull-up wiring described in Section 2 – this was first proven out on the Uno's own `INPUT_PULLUP` mode before being carried over unchanged to the standalone build.

6.5 Phase I Validation Checklist

Check	Result
All seven segments map to the correct physical segment on every digit	✓ Pass
No visible flicker at the chosen multiplex refresh rate	✓ Pass
No cross-talk / ghosting between adjacent digits	✓ Pass
Hour and minute buttons increment correctly and roll over ($23 \rightarrow 0$, $59 \rightarrow 0$)	✓ Pass
Start/Mode button switches to Seconds Mode and times out back to Normal Mode	✓ Pass
Time stays accurate over an extended soak test (no drift from missed `millis()` ticks)	✓ Pass

Table 4: Validation results before migrating off the Arduino Uno.

7 Phase II — Migrating to the Standalone ATmega328P-PU

7.1 Why Remove the Arduino Board

With the logic validated, the Arduino Uno board itself – its USB interface chip, voltage regulator, and supporting passives – is no longer needed. Only the microcontroller it carries is required for the final product.

Parameter	Value
Supply voltage	5 V
Clock speed	16 MHz (external crystal)
Flash memory	32 KB
SRAM	2 KB
EEPROM	1 KB
Usable GPIO	23 pins
Package	PDIP-28

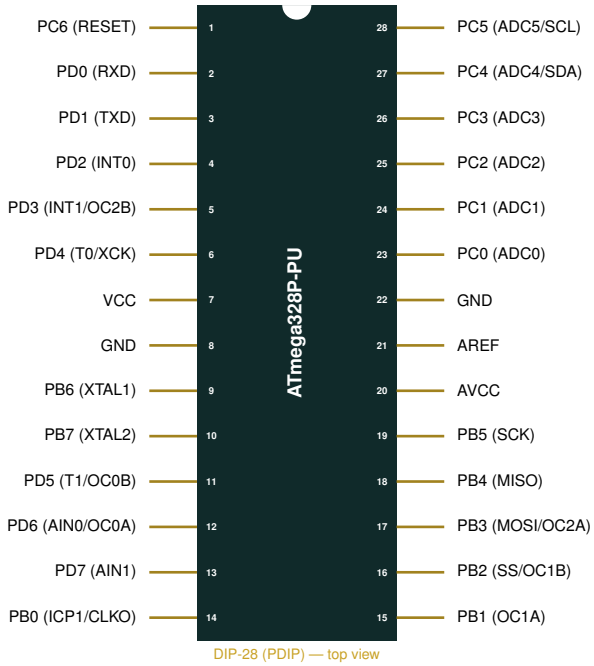
Table 5: ATmega328P-PU key specifications.

Advantages of the standalone approach

Compact – no board outline to fit around; **cheaper** – the bare chip costs a fraction of a full Uno; **professional** – mirrors how this chip is actually used in shipped products; **independent** – the finished clock has no dependency on a development board it would otherwise need to keep plugged in.

7.2 Learning the Pinout

Working with the bare DIP package means every connection that the Arduino board used to make invisibly – crystal, reset, power – must now be wired by hand. Figure 7.2 maps every physical pin of the 28-pin package to its AVR port name and primary function.



DIP-28 (PDIP) — top view

Pins reserved before any GPIO is even assigned

Of the 28 physical pins, **5 are already committed** before a single I/O line is allocated to the display: pins 7 & 8 (VCC/GND), pin 1 (RESET), and pins 9 & 10 (XTAL1/XTAL2, consumed by the crystal). Pin 20 (AVCC) and pin 22 (GND) are also tied to the supply rails. That leaves

the three full ports – B, C, and D – minus those reservations, i.e. **23 usable GPIO pins**, exactly matching the datasheet figure in Table 5.

7.3 Arduino Pin to Physical Pin Cross-Reference

The Arduino IDE refers to pins by their board-silkscreen names (`D2`, `A0`, etc.), but the bare chip only understands port/pin terms. Table 6 translates every pin used in this project from its Arduino name to its physical DIP location, so the same wiring used on the Uno (Section 6) can be reproduced on the breadboard around the bare chip.

Arduino Pin	AVR Port Pin	Physical Pin #	Used For
D2	PD2	4	Segment a
D3	PD3	5	Segment b
D4	PD4	6	Segment c
D5	PD5	11	Segment d
D6	PD6	12	Segment e
D7	PD7	13	Segment f
D8	PB0	14	Segment g
D9	PB1	15	Digit 1 select
D10	PB2	16	Digit 2 select
D11	PB3	17	Digit 3 select
D12	PB4	18	Digit 4 select
A0	PC0	23	Button 1 (Hour)
A1	PC1	24	Button 2 (Minute)
A2	PC2	25	Button 3 (Start/Mode)

Table 6: Project-specific Arduino-pin to physical-DIP-pin cross-reference.

7.4 Rebuilding the Minimum Operating Circuit

Once the bare chip sits on the breadboard, the four support networks designed and calculated in Section 5 – the crystal oscillator (Figure 2), the decoupling capacitors (Figure 3), and the reset pull-up (Figure 4) – are wired directly onto the physical pins identified in Figure 7.2: pins 9/10 for the crystal, pins 7/20 for VCC/AVCC decoupling, and pin 1 for the reset pull-up. Only once this minimum operating circuit is in place and verified (chip draws idle current, no overheating, reset line measured at 5 V) does wiring of the display and buttons begin.

8 Phase III — Bootloader Burning via Arduino as ISP

8.1 Why a Fresh Chip Needs a Bootloader

A brand-new ATmega328P-PU from the factory has no Arduino bootloader and no fuse bits configured for a 16 MHz external crystal – it is generic silicon. Before it will accept sketches uploaded the "Arduino way," it must be burned with the Arduino Uno bootloader and have its clock-source fuses set correctly. A second Arduino Uno, reflashed to act as an **AVR ISP (In-System Programmer)**, performs this job – no dedicated programmer hardware needs to be purchased.

8.2 Arduino-as-ISP Wiring

Arduino Uno (Programmer)	ATmega328P-PU (Target)
D10	RESET (pin 1)
D11	MOSI (pin 17)
D12	MISO (pin 18)
D13	SCK (pin 19)
5V	VCC (pin 7)
GND	GND (pin 8)

Table 7: Arduino-as-ISP to target-chip wiring.

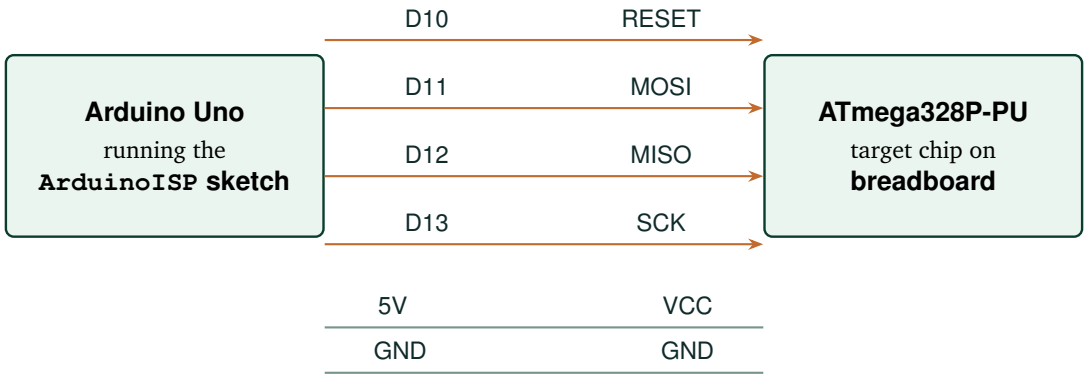


Figure 7: Arduino-as-ISP connection used to burn the bootloader and, later, the clock firmware itself.

8.3 Bootloader Burning Procedure

1. Open the Arduino IDE, load the built-in **File** → **Examples** → **11.ArduinoISP** → **ArduinoISP** sketch, and upload it to the *programmer* Uno (with its own USB cable connected, board still selected as "Arduino Uno").
2. Wire the programmer Uno to the target ATmega328P-PU on the breadboard exactly as in Table 7 / Figure 7. The target chip must already have its minimum operating circuit (Section 7) in place.
3. In the IDE, set **Tools** → **Board** → **Arduino Uno** (this selects the correct bootloader image and fuse settings for the target chip) and **Tools** → **Programmer** → **Arduino as ISP**.
4. Run **Tools** → **Burn Bootloader**. The IDE uses the programmer Uno to write the bootloader and set the fuses (clock source = external crystal, brown-out detection, etc.) on the target chip.
5. Confirm a "Done burning bootloader" message with no errors before proceeding.

If the target chip's crystal and capacitors (Section 5.1) are not yet wired up at this point, bootloader burning will fail or hang – the ISP process depends on the target chip actually running at the expected clock speed once fuses referencing an external crystal are set.

9 Phase IV — Uploading the Clock Firmware

9.1 Programming Logic Overview

With a bootloader in place, the chip behaves like a standard Arduino Uno from the IDE's point of view. The firmware's timekeeping core avoids any dependency on a hardware RTC, instead deriving hours, minutes, and seconds purely from the AVR's free-running millisecond timer:

Listing 1: Timekeeping core (simplified).

```
unsigned long lastTick = 0;
int seconds = 0, minutes = 0, hours = 0;

void updateClock() {
  if (millis() - lastTick >= 1000) {   // one real second elapsed
    lastTick += 1000;
    seconds++;
    if (seconds >= 60) { seconds = 0; minutes++; }
    if (minutes >= 60) { minutes = 0; hours++; }
    if (hours   >= 24) { hours = 0; }
  }
}
```

The display routine runs independently and far more frequently, continuously cycling through the four digits (Section 6.3) so that whichever values `hours/minutes` (or `minutes/seconds`, in Seconds Mode) currently hold are always shown without any visible flicker.

9.2 Source Code Repository

Full Arduino Sketch

The complete, uploadable Arduino sketch for this project – segment mapping, multiplexing routine, button handling, and both display modes – is published here:

`https://github.com/samikshas1118/IITH-SRFP/blob/main/Digital_Clock_Code`

9.3 Uploading the Firmware to the Bare Chip

The same Arduino-as-ISP wiring used for bootloader burning (Figure 7) doubles as the upload path for the actual clock sketch – the target chip has no USB/serial interface of its own, so every future firmware update must go through this same ISP route (or, alternatively, via a separate FTDI/USB-to-serial adapter wired to the chip's RX/TX pins, if preferred):

1. Open the clock sketch from the repository above in the Arduino IDE.
2. With **Board = Arduino Uno** and **Programmer = Arduino as ISP** still selected, choose **Sketch → Upload Using Programmer** (*not* the regular Upload button, since the target has no bootloader-driven serial path through the programmer Uno).
3. Watch for "Done uploading" with no `avrdude` errors.

10 Phase V — Full-Circuit Breadboard Validation

Before committing to solder, the *entire* standalone circuit – chip, crystal network, decoupling, reset, all four displays, both colon LEDs, and all three buttons – was reassembled and re-tested on a

breadboard, powered exactly as the final product would be (5 V USB module, no Arduino board attached at all).

Test Case	Result
Chip powers up and runs without the programmer Uno attached	✓ Pass
All four digits display correctly in Normal Mode (HH:MM)	✓ Pass
Seconds Mode (MM:SS) triggers on long-press and times out correctly	✓ Pass
Hour / Minute buttons set time correctly, including roll-over	✓ Pass
Colon LEDs are steady and clearly visible	✓ Pass
No flicker, ghosting, or dim digits at operating voltage	✓ Pass
RESET line measured at a stable 5 V during normal run	✓ Pass
No unexpected resets over an extended power-on soak test	✓ Pass

Table 8: Full-circuit breadboard validation results, immediately prior to soldering.

■ Gate before soldering

Every row in Table 8 had to pass before any component touched the dot board. Soldering is, by design, the *least* reversible step in this entire workflow – so it is deliberately the last one to happen.

11 Phase VI — Soldering & Permanent Assembly

11.1 From Breadboard to Solder

Once Table 8 was fully green, the validated layout was transferred, connection for connection, onto a perforated dot board for permanent assembly.

11.2 Soldering Precautions

- Keep crystal leads and the two 22 pF capacitor leads as **short as possible** – long leads here add stray inductance/capacitance that can destabilise oscillation.
- Double-check **polarity** on every polarised part (LEDs, electrolytic capacitors if used) before soldering – a reversed LED simply won't light, but a reversed electrolytic can fail violently.
- Verify the **chip orientation** (notch / pin-1 dot, Figure 7.2) before inserting it into a socket – never solder the chip directly without a socket, both to protect it from soldering heat and to allow future reprogramming or replacement.
- Inspect every joint for **solder bridges** between adjacent pins, especially across the closely-spaced DIP-28 socket pins.

- Use a fine-tip iron and **check continuity** with a multimeter on every power and ground run before first power-up.
- Power up for the first time **without** the microcontroller inserted, verify 5 V and 0 V land where expected, then insert the chip.

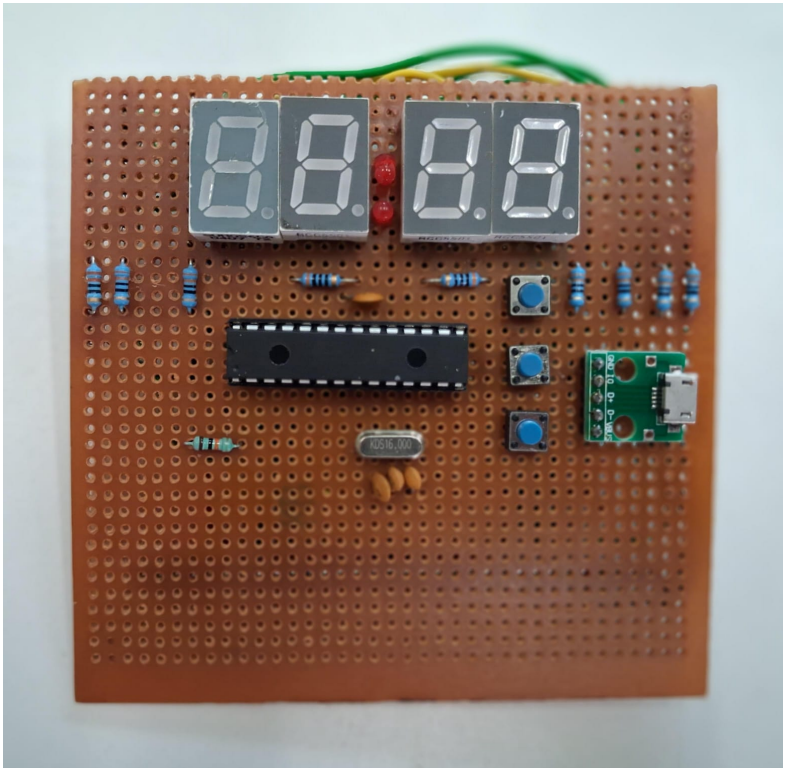


Figure 8: The soldered standalone clock board – ATmega328P-PU (socketed), crystal network, displays, buttons, and power module assembled on dot board.

11.3 Post-Solder Bring-Up Checklist

Check	Result
Visual inspection: no solder bridges, no cold joints	✓ Pass
Continuity confirmed on all VCC and GND runs	✓ Pass
5 V measured correctly before chip insertion	✓ Pass
Chip inserted in correct orientation, board powers up cleanly	✓ Pass
Full display & button re-test identical to Table 8	✓ Pass

Table 9: Bring-up checks performed immediately after soldering.

12 Phase VII — Mechanical Enclosure Design (Fusion 360)

12.1 Design Goals

With a working soldered board in hand, a custom enclosure was modelled in **Autodesk Fusion 360** to:

- Physically protect the board, solder joints, and wiring;
- Present a clean, finished, non-prototype appearance;
- Provide a display window so the seven-segment digits remain clearly visible;
- Provide access cut-outs aligned to the three push buttons;
- Assemble and disassemble **without screws**, using a snap-fit joint between the two shells.

12.2 Snap-Fit Mechanism

Rather than threaded fasteners, the enclosure's top and bottom shells lock together using a series of cantilever snap hooks moulded into the bottom shell, which engage matching lips/slots printed into the inside wall of the top shell (Figure 9).

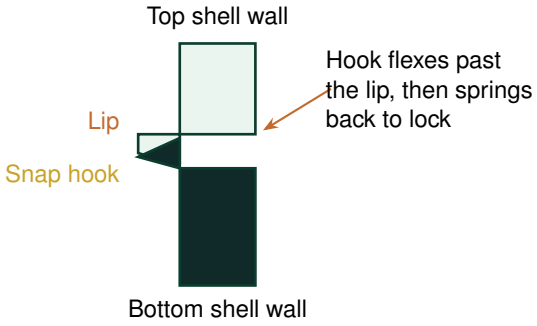


Figure 9: Snap-fit joint cross-section: a cantilever hook on the bottom shell deflects past, then springs back behind, a matching lip on the top shell.

Why snap-fit over screws

A screwed enclosure needs bosses, threaded inserts or self-tapping pilot holes, and visible fastener heads. A snap-fit shell needs none of that: it assembles with a firm press, disassembles by releasing the hooks with a thin tool, and leaves a cleaner, hardware-free exterior – at the cost of needing slightly more careful tolerancing (typically 0.2–0.3 mm clearance between hook and lip) so the joint is snug without being so tight that the PLA hook snaps off during assembly.

12.3 CAD Renders

Figure 10 and Figure 11 show the completed top and bottom shells as modelled in Fusion 360.

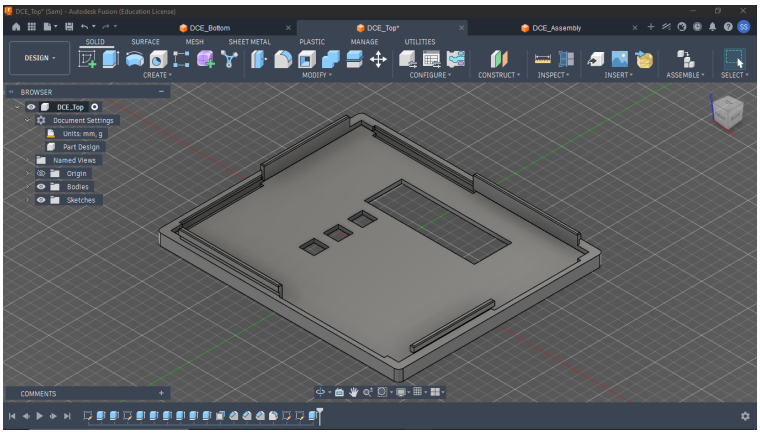


Figure 10: Top shell – Fusion 360 CAD render, showing the display window and button access cut-outs.

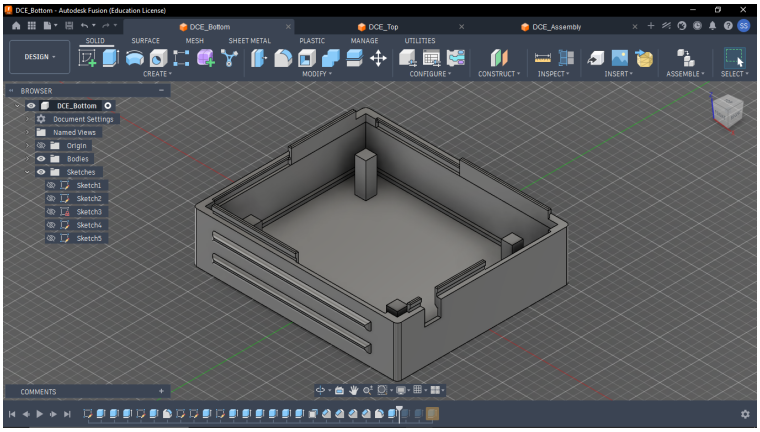


Figure 11: Bottom shell – Fusion 360 CAD render, showing the internal snap-fit hooks and board standoffs.

13 Phase VIII — Slicing & 3D Printing

13.1 From CAD Model to Printed Part

Both shells were modelled as separate bodies in Fusion 360, each exported as an individual `.STL` mesh, then imported into **Creality's slicer software** to be converted into machine-readable G-code.

13.2 Slicing Parameters Considered

Parameter	Consideration
Layer height	Finer layers improve the visual finish of the visible enclosure surfaces at the cost of print time.
Infill	Moderate infill is sufficient – the shells are not load-bearing structural parts, just protective housings.
Orientation	Each shell oriented to print its snap-fit hooks/lips along the strongest layer direction, minimising the chance of a hook snapping under flex.
Supports	Used selectively wherever an overhang (e.g. the display window lip, button bosses) would otherwise droop or print poorly bridging open air.

Table 10: Key slicing parameters configured before generating G-code.

14 Results & Final Outcome

The completed device is a fully standalone, enclosed digital clock built around a bare ATmega328P-PU rather than an Arduino board. It reliably displays HH:MM in Normal Mode, switches to MM:SS in Seconds Mode on demand, accepts hour/minute/start-mode input from three buttons, and runs

continuously from a simple 5 V USB supply – all inside a custom, snap-fit, 3D-printed shell with no exposed fasteners.

Outcome Summary

Idea → Calculated component selection → Arduino Uno breadboard prototype → Standalone ATmega328P-PU → ISP bootloader & firmware burning → Full-circuit re-validation → Soldered assembly → Fusion 360 snap-fit enclosure → 3D-printed final product.

15 Skills Demonstrated

✓ Embedded systems design with a bare microcontroller (no development board in the final product)

✓ Digital electronics: oscillator, decoupling, and reset network calculation

✓ Display multiplexing and persistence-of-vision driven design

✓ Microcontroller programming (Arduino C/C++, `millis()`-based scheduling)

✓ AVR ISP programming and bootloader/fuse management

✓ Hardware debugging across two breadboard iterations

✓ Soldering and permanent through-hole assembly

✓ Parametric CAD modelling (Fusion 360) and snap-fit mechanical design

✓ Slicing and FDM 3D-printing workflow (Creality Slicer)

16 Future Scope

- **Real-Time Clock (RTC) module** (e.g. DS1307/DS3231) to retain accurate time through power loss, removing reliance on `millis()` drift correction and manual re-setting after every power cycle.
- **Battery backup** (coin cell + RTC) so the clock keeps time even when unplugged.
- **Alarm and buzzer functionality** using one of the chip's remaining free GPIO pins.
- **Temperature/date display** as an additional rotating mode, using the same multiplexed digits.
- **Brightness control** via PWM-dimmed segment drive, using a light-dependent resistor (LDR) for ambient-light-adaptive brightness.
- **Digit-driver transistors** (Section 5.4) to formally bring every GPIO pin's current within the AVR's recommended continuous rating.
- **PCB version** replacing the dot board with a custom-etched or fabricated PCB for a more compact, more reliable final assembly.

17 Conclusion

This project set out to answer a simple question – what does it actually take to turn an Arduino Uno sketch into a real, standalone product? – and answered it by walking the entire path: justifying every component with a calculation, proving logic safely on a breadboard before committing to solder, learning to read and wire a bare 28-pin chip directly, reusing available hardware (a second Uno)

as a programmer rather than buying new tools, and finishing with a properly designed, snap-fit 3D-printed enclosure rather than a bare board on a desk. The result is more than "a clock that tells time" – it is a complete, documented demonstration of the idea-to-product workflow that underlies real embedded hardware development.

18 References & Resources

- Arduino Sketch (this project): https://github.com/samikshas1118/IITH-SRFP/blob/main/Digital_Clock_Code
- ATmega328P-PU Datasheet – Microchip/Atmel
- Arduino IDE built-in example: File → Examples → 11.ArduinoISP → ArduinoISP
- Autodesk Fusion 360 – CAD modelling software used for the enclosure
- Creality Slicer – slicing software used to generate G-code for FDM printing